Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ N 6

Тема: «Хеш-функции и имитовставка: контроль целостности данных»

Дисциплина: «Защита информации»

Вариант: 1 (пример запуска)

Студент ИУ7-72Б _____ Ю.А. Попов
(Группа) (Подпись, дата) (И. О. Фамилия)

Преподаватель _____ Ю.С. Руденкова
(Подпись, дата) (И. О. Фамилия)

Москва, 2026 г.

1 Цель работы и теоретические сведения

Цель работы — исследовать свойства криптографических хеш-функций, применение имитовставки для контроля целостности и использование PBKDF2 для хранения паролей.

Хеш-функция преобразует сообщение произвольной длины в дайджест фиксированного размера. При проверке целостности полученный дайджест сравнивают с эталонным. Сам по себе хеш не подтверждает отправителя: противник, изменив данные, может вычислить новый дайджест.

К криптографической хеш-функции предъявляют требования стойкости к поиску прообраза, второго прообраза и коллизии. Для идеального n-битного хеша поиск прообраза требует порядка 2^n вычислений, а поиск коллизии — порядка $2^{n/2}$ из-за парадокса дней рождения. Лавинный эффект означает, что изменение одного бита входа меняет в среднем половину бит дайджеста.

CRC применяется для обнаружения случайных ошибок передачи. Её алгебраические свойства позволяют намеренно изменить данные и подобрать компенсирующую поправку, поэтому CRC не обеспечивает защиту от подмены.

Имитовставка HMAC вычисляется по сообщению и секретному ключу. Проверяющий с тем же ключом может обнаружить изменение сообщения. В отличие от простого хеширования ключа вместе с сообщением HMAC применяет два вложенных хеширования и специальные дополнения ключа.

Для хранения паролей используют независимую случайную соль и медленную функцию вывода ключа, например PBKDF2. Соль препятствует использованию общих предвычисленных таблиц для разных записей, а многократные итерации повышают стоимость каждой попытки перебора.

2 Исходные данные

Номер варианта преподавателем не указан; ниже приведён пример запуска для варианта 1. Сообщение: `user=student01&role=user`; ключ: `5103256fec974d6c` (8 байт). Для HMAC используется SHA-256 с усечением до 96 бит, для поиска коллизий — SHA-256 с усечением до 24 бит.

SHA-256 сообщения:

`6ec6391bf3f878938fc9f779f1407500beff9fed4d3b74ece82880b0b9ae9aa9`

3 Выполнение работы

3.1 Подготовка исходных данных

Функция формирования данных варианта:

```python
def variant(n: int) -> dict:
    if not 1 <= n <= 20:
        raise ValueError('Номер варианта должен быть от 1 до 20')
    i = (n - 1) % 5
    key_len = [8, 12, 16, 20, 24][i]
    return {
        'M': f'user=student{n:02d}&role=user'.encode(),
        'K': hashlib.sha256(f'lab6-key-{n}'.encode()).digest()[:key_len],
        't_bits': [96, 112, 128, 160, 192][i],
        'collision_bits': [24, 28, 32, 36, 40][i],
        'digest': ['sha256', 'sha512', 'sha3_256', 'blake2b', 'sha384'][i],
    }
```

Результат: сообщение и ключ приведены в разделе 2, SHA-256 сообщения — `6ec6391bf3f878938fc9f779f1407500beff9fed4d3b74ece82880b0b9ae9aa9`. Вывод: параметры однозначно определяются номером варианта.

3.2 Лавинный эффект

Функции расстояния Хэмминга и эксперимента:

```python
def hamming(a: bytes, b: bytes) -> int:
    if len(a) != len(b):
        raise ValueError('Длины строк должны совпадать')
    return sum((x ^ y).bit_count() for x, y in zip(a, b))
```

```python
def avalanche_results(msg: bytes, trials: int = 256) -> list:
    if not msg or trials < 256:
        raise ValueError('Требуются непустое сообщение и не менее 256 испытаний')
    original = hashlib.sha256(msg).digest()
    results = []
    for i in range(max(trials, len(msg) * 8)):
        changed = bytearray(msg)
        bit = i % (len(msg) * 8)
        changed[bit // 8] ^= 1 << (bit % 8)
        results.append(hamming(original, hashlib.sha256(changed).digest()))
    return results
```

```python
def avalanche(msg: bytes, trials: int = 256) -> float:
    results = avalanche_results(msg, trials)
```

```
    return sum(results) / len(results)
```

Результат: за 256 испытаний среднее число изменившихся бит составило 128,32 из 256. Гистограмма: каждый символ # соответствует одному испытанию.

```
109: # (1)
113: ### (3)
114: ## (2)
115: ##### (5)
116: ########## (10)
117: #### (4)
118: ### (3)
119: ###### (6)
120: ######## (8)
121: #### (4)
122: ########## (10)
123: ############ (12)
124: ########## (10)
125: ######## (8)
126: ######## (8)
127: #################### (20)
128: ############ (12)
129: ######################## (24)
130: ############ (12)
131: ###### (6)
132: ########### (11)
133: ###### (6)
134: ############## (14)
135: ############## (14)
136: ######## (8)
137: #### (4)
138: ####### (7)
139: ####### (7)
140: ##### (5)
141: ### (3)
142: ###### (6)
143: ## (2)
144: # (1)
```

Вывод: среднее близко к половине длины хеша; каждый бит сообщения проверен, а часть битов повторена для достижения 256 испытаний.

3.3 Слабость CRC

```
def crc_is_linear() -> bool:
    for _ in range(100):
        a, b, c = (secrets.token_bytes(32) for _ in range(3))
        combined = bytes(x ^ y ^ z for x, y, z in zip(a, b, c))
        if zlib.crc32(combined) != zlib.crc32(a) ^ zlib.crc32(b) ^ zlib.crc32(c):
            return False
    return True
```

Результат: равенство CRC для трёх строк одинаковой длины подтвердилось в 100 испытаниях (`True`). Вывод: аффинное свойство CRC позволяет вычислять компенсирующие изменения, поэтому CRC не защищает от преднамеренной подмены.

3.4 Коллизия усечённого хеша

```python
def truncated(msg: bytes, bits: int) -> bytes:
    if not 1 <= bits <= 256:
        raise ValueError('Длина усечения должна быть от 1 до 256 бит')
    result = bytearray(hashlib.sha256(msg).digest()[:(bits + 7) // 8])
    if bits % 8:
        result[-1] &= 0xff << (8 - bits % 8)
    return bytes(result)
```

```python
def birthday_collision(bits: int):
    seen = {}
    attempts = 0
    while True:
        msg = secrets.token_bytes(16)
        digest = truncated(msg, bits)
        attempts += 1
        previous = seen.get(digest)
        if previous is not None and previous != msg:
            return previous, msg, digest, attempts
        seen[digest] = msg
```

Таблица 3.1 — Число попыток поиска коллизии в пяти запусках

1	2	3	4	5
8743	12783	2557	3538	4163

Пример коллизии: сообщения `2ad6c85f7ad871162b7995e52aa4d1bd` и `a945e284768205edb2a1796ef632fa65` дали усечённый хеш `ec736d`. Среднее составило 6356,8 попытки при оценке порядка $2^{24/2} = 4096$. Вывод: результат имеет тот же порядок; отклонение зависит от случайности поиска и малого числа запусков.

3.5 Имитовставка

```python
def naive_mac(key: bytes, msg: bytes, digest: str = 'sha256') -> str:
    return hashlib.new(digest, key + msg).hexdigest()
```

```python
def hmac_mac(key: bytes, msg: bytes, digest: str = 'sha256', bits: int = 256) -> str:
    return hmac.new(key, msg, digest).digest()[:bits // 8].hex()
```

```python
def hmac_verify(key: bytes, msg: bytes, tag: str, digest: str = 'sha256', bits: int = 256) -> bool:
```

```
2     return hmac.compare_digest(hmac_mac(key, msg, digest, bits), tag)
```

Результат: наивный MAC — `a0f21a9db5fd0864527daa9390986d8267ab68b53d9d72792f592bcd1b36ebe7`, HMAC — `20a06709154809ce97c56773`. Вывод: HMAC формирует ключевой тег; `compare_digest` уменьшает утечку информации о совпадающем префиксе тега через время сравнения.

3.6 Контроль целостности

```
1 def tampered_demo(key: bytes, msg: bytes, digest: str = 'sha256', bits: int = 256) -> bool:
2     if b'role=user' not in msg:
3         raise ValueError('В сообщении отсутствует role=user')
4     tag = hmac_mac(key, msg, digest, bits)
5     changed = msg.replace(b'role=user', b'role=admin')
6     print('Изменённое сообщение с новым тегом:', hmac_verify(key, changed, hmac_mac(key, changed, digest,
      bits), digest, bits))
7     return hmac_verify(key, changed, tag, digest, bits)
```

Результат: проверка изменённого сообщения со старым тегом — `False`, с новым тегом — `True`. Вывод: изменение обнаружено, однако владелец ключа может подписать изменённое сообщение заново.

3.7 Хранение пароля

```
1 def store_password(password: str) -> dict:
2     salt = secrets.token_bytes(16)
3     iterations = 100_000
4     return {'salt': salt, 'hash': hashlib.pbkdf2_hmac('sha256', password.encode(), salt, iterations),
      'iterations': iterations}
```

```
1 def check_password(password: str, record: dict) -> bool:
2     result = hashlib.pbkdf2_hmac('sha256', password.encode(), record['salt'], record['iterations'])
3     return hmac.compare_digest(result, record['hash'])
```

Результат: записи одинакового пароля различаются; пароль успешно проверен. Соль каждой записи содержит 16 байт, число итераций — 100 000. Измеренное время проверки — около 0,010 с, оценка последовательного перебора 10 млн паролей — 1,16 суток. Вывод: случайная соль различает записи, а итерации повышают стоимость перебора; скорость зависит от оборудования и параллелизма.

3.8 Дополнительное задание: ручной HMAC

```python
def manual_hmac(key: bytes, msg: bytes, digest: str = 'sha256', bits: int = 256) -> str:
    block_size = hashlib.new(digest).block_size
    if len(key) > block_size:
        key = hashlib.new(digest, key).digest()
    key = key.ljust(block_size, b'\0')
    inner = bytes(x ^ 0x36 for x in key)
    outer = bytes(x ^ 0x5c for x in key)
    inner_hash = hashlib.new(digest, inner + msg).digest()
    return hashlib.new(digest, outer + inner_hash).digest()[:bits // 8].hex()
```

Результат: ручная реализация совпала с результатом стандартного модуля (`True`). Вывод: подготовка ключа и вложенные хеширования дают тот же тег для выбранного варианта.

4 Контрольные вопросы

1) **Чем криптографическая хеш-функция отличается от обычной контрольной суммы?** Криптографический хеш рассчитан на сопротивление намеренному подбору, а CRC — на обнаружение случайных ошибок.
2) **Сформулируйте три свойства стойкости хеш-функции. Какое из них достигается легче всего и почему?** Это стойкость к поиску прообраза, второго прообраза и коллизии. Легче найти коллизию: из-за парадокса дней рождения для n-битного хеша нужно порядка $2^{n/2}$ попыток.
3) **Что такое лавинный эффект и зачем он нужен?** Изменение одного бита входа меняет в среднем половину бит результата, затрудняя установление простых связей между входом и хешем.
4) **Почему для стойкости 128 бит к коллизиям нужен хеш длиной 256 бит?** Поиск коллизии требует порядка $2^{n/2}$ вычислений, поэтому при $n = 256$ оценка равна 2^{128}.
5) **Почему CRC нельзя использовать для защиты от умышленного изменения данных?** Аффинная структура CRC позволяет вычислить компенсирующее изменение данных.

6) **Чем имитовставка отличается от хеш-функции и от электронной подписи?** Хеш не использует ключ; MAC требует общего секретного ключа; подпись создаётся закрытым ключом и проверяется открытым.
7) **Почему конструкция hash(K || M) небезопасна, а HMAC — безопасна?** Для ряда хеш-функций первый вариант допускает атаку удлинения сообщения. В HMAC вложенное хеширование предотвращает эту атаку; стойкость зависит от применяемой функции и секретности ключа.
8) **Зачем при хранении паролей нужны соль и медленная функция? От каких атак защищает каждая из них?** Уникальная соль препятствует общим предвычислениям и радужным таблицам; медленная функция повышает стоимость каждой попытки словарного или полного перебора.
9) **Что такое аутентифицированное шифрование и чем оно лучше раздельного шифрования и вычисления имитовставки?** Оно объединяет конфиденциальность и проверку целостности в единой схеме, снижая риск ошибок сочетания алгоритмов. Корректно реализованное раздельное сочетание также может быть безопасным.
10) **Приведите пример реального инцидента, связанного со взломом хеш-функции (из лекции).** Проект SHAttered в 2017 году продемонстрировал два разных PDF с одинаковым SHA-1. Материалы лекции не предоставлены, поэтому соответствие примера лекции не проверено.

5 Вывод

Исследованы лавинный эффект и коллизии SHA-256, проверена слабость CRC против подмены, реализованы HMAC и хранение паролей с PBKDF2. Эксперименты подтвердили обнаружение изменения сообщения и различие записей одинаковых паролей.

ПРИЛОЖЕНИЕ А. Текст программы

```python
"""Лабораторная работа 6. Хеш-функции и имитовставка."""
import hashlib
import hmac
import os
import secrets
import zlib

VARIANT = 0  # Укажите вариант от 1 до 20 или введите его при запуске.


def variant(n: int) -> dict:
    if not 1 <= n <= 20:
        raise ValueError('Номер варианта должен быть от 1 до 20')
    i = (n - 1) % 5
    key_len = [8, 12, 16, 20, 24][i]
    return {
        'M': f'user=student{n:02d}&role=user'.encode(),
        'K': hashlib.sha256(f'lab6-key-{n}'.encode()).digest()[:key_len],
        't_bits': [96, 112, 128, 160, 192][i],
        'collision_bits': [24, 28, 32, 36, 40][i],
        'digest': ['sha256', 'sha512', 'sha3_256', 'blake2b', 'sha384'][i],
    }


def hamming(a: bytes, b: bytes) -> int:
    if len(a) != len(b):
        raise ValueError('Длины строк должны совпадать')
    return sum((x ^ y).bit_count() for x, y in zip(a, b))


def avalanche_results(msg: bytes, trials: int = 256) -> list:
    if not msg or trials < 256:
        raise ValueError('Требуются непустое сообщение и не менее 256 испытаний')
    original = hashlib.sha256(msg).digest()
    results = []
    for i in range(max(trials, len(msg) * 8)):
        changed = bytearray(msg)
        bit = i % (len(msg) * 8)
        changed[bit // 8] ^= 1 << (bit % 8)
        results.append(hamming(original, hashlib.sha256(changed).digest()))
    return results


def avalanche(msg: bytes, trials: int = 256) -> float:
    results = avalanche_results(msg, trials)
    return sum(results) / len(results)


def crc_is_linear() -> bool:
    for _ in range(100):
        a, b, c = (secrets.token_bytes(32) for _ in range(3))
        combined = bytes(x ^ y ^ z for x, y, z in zip(a, b, c))
        if zlib.crc32(combined) != zlib.crc32(a) ^ zlib.crc32(b) ^ zlib.crc32(c):
            return False
    return True


def truncated(msg: bytes, bits: int) -> bytes:
    if not 1 <= bits <= 256:
        raise ValueError('Длина усечения должна быть от 1 до 256 бит')
    result = bytearray(hashlib.sha256(msg).digest()[:(bits + 7) // 8])
    if bits % 8:
```

```
        result[-1] &= 0xff << (8 - bits % 8)
    return bytes(result)


def birthday_collision(bits: int):
    seen = {}
    attempts = 0
    while True:
        msg = secrets.token_bytes(16)
        digest = truncated(msg, bits)
        attempts += 1
        previous = seen.get(digest)
        if previous is not None and previous != msg:
            return previous, msg, digest, attempts
        seen[digest] = msg


def naive_mac(key: bytes, msg: bytes, digest: str = 'sha256') -> str:
    return hashlib.new(digest, key + msg).hexdigest()


def hmac_mac(key: bytes, msg: bytes, digest: str = 'sha256', bits: int = 256) -> str:
    return hmac.new(key, msg, digest).digest()[:bits // 8].hex()


def hmac_verify(key: bytes, msg: bytes, tag: str, digest: str = 'sha256', bits: int = 256) -> bool:
    return hmac.compare_digest(hmac_mac(key, msg, digest, bits), tag)


def manual_hmac(key: bytes, msg: bytes, digest: str = 'sha256', bits: int = 256) -> str:
    block_size = hashlib.new(digest).block_size
    if len(key) > block_size:
        key = hashlib.new(digest, key).digest()
    key = key.ljust(block_size, b'\0')
    inner = bytes(x ^ 0x36 for x in key)
    outer = bytes(x ^ 0x5c for x in key)
    inner_hash = hashlib.new(digest, inner + msg).digest()
    return hashlib.new(digest, outer + inner_hash).digest()[:bits // 8].hex()


def tampered_demo(key: bytes, msg: bytes, digest: str = 'sha256', bits: int = 256) -> bool:
    if b'role=user' not in msg:
        raise ValueError('В сообщении отсутствует role=user')
    tag = hmac_mac(key, msg, digest, bits)
    changed = msg.replace(b'role=user', b'role=admin')
    print('Изменённое сообщение с новым тегом:', hmac_verify(key, changed, hmac_mac(key, changed, digest, bits),
    digest, bits))
    return hmac_verify(key, changed, tag, digest, bits)


def store_password(password: str) -> dict:
    salt = secrets.token_bytes(16)
    iterations = 100_000
    return {'salt': salt, 'hash': hashlib.pbkdf2_hmac('sha256', password.encode(), salt, iterations), 'iterations':
    iterations}


def check_password(password: str, record: dict) -> bool:
    result = hashlib.pbkdf2_hmac('sha256', password.encode(), record['salt'], record['iterations'])
    return hmac.compare_digest(result, record['hash'])


if __name__ == '__main__':
    try:
        number = VARIANT or int(input('Номер варианта (1–20): '))
        v = variant(number)
```

```
    except ValueError as exc:
        print('Ошибка:', exc)
        raise SystemExit(1)
    msg, key = v['M'], v['K']
    digest, bits = v['digest'], v['t_bits']
    print('Вариант:', number, '| M:', msg.decode(), '| K:', key.hex())
    print('SHA-256:', hashlib.sha256(msg).hexdigest())
    results = avalanche_results(msg)
    print(f'Лавинный эффект: {len(results)} испытаний, среднее {sum(results)/len(results):.2f}')
    print('Гистограмма (изменившихся бит: количество):')
    for value in sorted(set(results)):
        print(f'{value}: {"#" * results.count(value)} ({results.count(value)})')
    print('Равенство CRC для строк одинаковой длины:', crc_is_linear())
    attempts = []
    for i in range(5):
        a, b, value, count = birthday_collision(v['collision_bits'])
        attempts.append(count)
        print(f'Коллизия {i+1}: {count} попыток, M1={a.hex()}, M2={b.hex()}, хеш={value.hex()}')
    print(f'Среднее: {sum(attempts)/5:.1f}; оценка: {2**(v["collision_bits"]/2):.1f}')
    tag = hmac_mac(key, msg, digest, bits)
    print('Наивный MAC:', naive_mac(key, msg, digest))
    print(f'HMAC ({digest}, {bits} бит):', tag)
    print('Ручной HMAC совпадает:', hmac.compare_digest(tag, manual_hmac(key, msg, digest, bits)))
    print('Подмена со старым тегом прошла:', tampered_demo(key, msg, digest, bits))
    first, second = store_password('example-password'), store_password('example-password')
    for i, record in enumerate((first, second), 1):
        print(f'Запись {i}: соль={record["salt"].hex()}, хеш={record["hash"].hex()}, итерации={record["iterations"]}')
    print('Записи различаются:', first != second)
    start = os.times().elapsed
    valid = check_password('example-password', first)
    elapsed = os.times().elapsed - start
    print(f'Пароль верен: {valid}, время проверки: {elapsed:.3f} с')
    print(f'Оценка перебора 10 млн паролей: {elapsed * 10_000_000 / 86400:.2f} суток')
```